

Stylistic Idiosyncrasies of Code Generated by Large Language Models

Weiye Li¹, Zezhong Chen², Jiang Fang^{3,2†}, Xuhong Zuo⁴, and Jiyan Sun^{5†}

¹*International School of BUPT, Beijing University of Posts and Telecommunications, Beijing, China*

²*Jianghuai Advance Technology Center, Hefei, China*

³*Electronic Engineering Institute, National University of Defense Technology*

⁴*School of Law and Criminal Justice, East China University of Political Science and Law, Shanghai, China*

⁵*Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China*

liweiyi@bupt.edu.cn, chenzezhong1513@163.com, jiangfang2025@gmail.com,

2221170089.ecupl@vip.163.com, sunjiyan@iie.ac.cn

Abstract—Large Language Models (LLMs) have demonstrated exceptional performance in source code generation across multiple programming languages. However, LLM-generated code often suffers from inconsistent style, structural redundancy, and potential security vulnerabilities. Therefore, source attribution for generated code is crucial for identifying the model origin, evaluating quality, and enhancing reliability. Due to strict syntactic constraints, code generated by different models for the same prompt exhibits much higher similarity than natural language text, causing existing text source attribution methods to degrade when applied to code attribution. To address this and analyze stylistic differences, we propose the Code Generation Source Detection (CGSD) framework. It first employs Supervised Contrastive Learning to obtain distinguishable code embeddings and trains a classifier to determine the code’s origin. CGSD then combines a similarity analysis module with the code style analyzer to deeply analyze coding styles of different models. Experiments on a self-constructed, multi-class dataset containing code from various mainstream LLMs show that CGSD achieves 93.25% accuracy, significantly outperforming existing baselines. This confirms that CGSD can both effectively capture model-specific generation patterns and reveal systematic stylistic differences across models.

Index Terms—Large Language Models, Code Generation, Supervised Contrastive Learning, Stylistic Analysis

I. INTRODUCTION

With the rapid advancement of large language models (LLMs) in code generation, output quality now approaches expert-level performance [1, 2], improving productivity across research and industry. Automated coding helps development teams focus on innovation and problem modeling rather than repetitive implementation.

Budget, compute, and expertise constraints push development teams toward cheaper or mixed-model pipelines, increasing exposure to security risks. About 26% of 452 GitHub Copilot snippets contained security vulnerabilities [3], underscoring the need for code provenance detection and stylistic auditing. Mixed-model codebases also suffer from divergent conventions—for instance, ChatGPT favors `number` while Gemini prefers `n`—which disrupts reuse and integration, motivating systematic source attribution.

Existing AIGC detection work has focused on detecting machine-generated text and code [4, 5, 6], but these efforts still fall short of fine-grained attribution across different generative models. Although some studies report strong results for text-level classification [7], applying those text-based classifiers directly to code leads to a marked drop in performance. LLM-generated programs are far more lexically and structurally homogeneous than natural language, so different model outputs often overlap in embedding space and are hard to separate. Moreover, code contains rich structural and semantic signals — syntax and control-flow patterns, domain-specific idioms, and stylistic markers in naming, comments, and layout — that general text models fail to capture well. These facts motivate domain-specific adaptations that enlarge inter-source margins and explicitly model code’s structural and stylistic features for reliable source attribution.

Motivated by the degradation of text-only classifiers on code and the need to pull apart these highly similar embeddings, we propose a Code Generation Source Detection (CGSD) framework that first leverages supervised contrastive learning to maximize source separation and then performs classification fine-tuning for effective multi-model code attribution. The framework consists of three integrated components that collectively enable accurate source detection and stylistic characterization. As illustrated in Figure 1, our approach integrates both training and analysis components:

- 1) **Embedding-based Classification:** Learn a discriminative projection via supervised contrastive learning using a projection head to obtain a separable embedding space, then fine-tune a classification head for multi-class attribution.
- 2) **Similarity Analysis:** Character- and word-level TF-IDF n -grams with cosine similarity, plus variable-naming Top- k statistics and pairwise heatmaps to quantify inter-model stylistic relationships.
- 3) **Code Style Analyzer:** Proposed in this paper, it covers six dimensions—naming, comments, syntax, consistency, complexity, and patterns. In practice we summarize each dimension with representative metrics to

†Corresponding author.

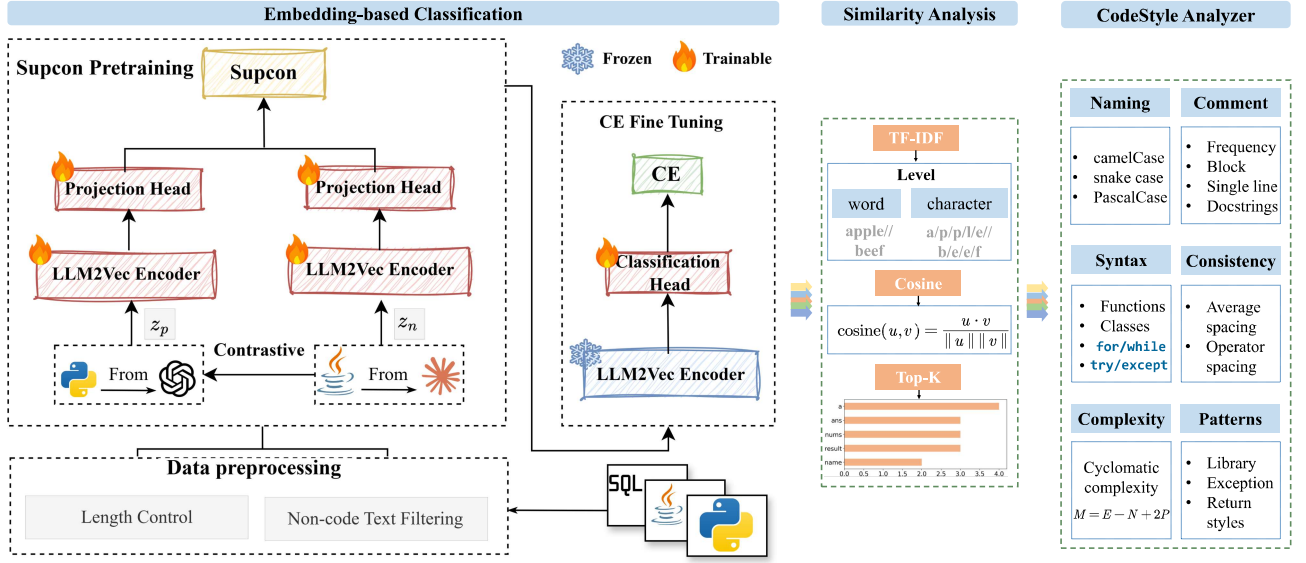


Fig. 1: Overview of the proposed CGSD framework: (1) Embedding-based classification employing supervised contrastive learning with classification fine-tuning for source attribution; (2) Pairwise similarity analysis using TF-IDF n-grams; (3) Code Style Analyzer for structural and stylistic metrics extraction. This integrated pipeline enables accurate source detection and systematic characterization of LLM code style.

explain inter-model style differences.

This integrated framework not only enables accurate source attribution but also supports comprehensive characterization of stylistic traits across different LLM-generated codebases. We evaluate CGSD on a self-constructed, multi-model dataset, validating its effectiveness in both code provenance detection and model-specific pattern analysis.

Experimental results show distinct stylistic patterns across different LLMs. Our approach achieves 93.25% attribution accuracy, outperforming LLM2Vec (60.20%) and BERT (58%) by nearly 33 percentage points.

Our main contributions are:

- We propose CGSD, a unified framework for LLM code source detection with embedding-based classification employing supervised contrastive learning and classification fine-tuning, integrating multi-class classification, similarity analysis, and stylistic quantification via Code Style Analyzer.
- CGSD achieves 93.25% attribution accuracy and reveals model-specific stylistic and structural patterns, improving interpretability and supporting code provenance applications. Performance far exceeds strong baselines, including a +33.05% absolute improvement over LLM2Vec.
- We construct a high-quality, curated multi-model dataset for code provenance, spanning multiple mainstream LLMs and programming languages, paired on shared prompts with preprocessing. Upon publication, we will open-source the dataset and tooling, establishing a strong benchmark for future research.

II. METHODOLOGY

Our approach integrates three components: embedding-based classification, code similarity analysis, and Code Style Analysis. Each module is indispensable to CGSD while reinforcing the others: classification supplies instance-level attribution and the supervised-contrastive embeddings that surface discriminative signals; similarity analysis contextualizes these attributions via TF-IDF/cosine similarity maps and identifier-frequency statistics that reveal cross-model proximity patterns; Code Style Analyzer then explains the drivers of separation with interpretable style and structure indicators. This forms an evidence chain—from signal, to attribution, to interpretation—supporting provenance analysis and comprehensive style characterization.

A. Embedding-based Classification

Large language models (LLMs) share similar architectures and training approaches—most rely on the Transformer framework [8], autoregressive learning objectives [9], and overlapping datasets such as GitHub and Stack Overflow. Despite these commonalities, it remains unclear whether different LLMs produce code with consistent stylistic patterns. To explore this, we design a code generation classification task to capture stylistic distinctions among LLM outputs.

1) *Data Cleaning*: To ensure high-quality training data, we employ a data cleaning strategy involving a script that selects high-quality samples from the collected responses. Specifically, we truncate each code sample to 512 tokens to align with the encoder’s input length limit, ensuring stable and

efficient training. Additionally, we filter out non-code comments embedded within the code, such as inline explanations or embedded documentation, which are irrelevant to code-style detection and can introduce confounding noise. This selection process filters out low-quality or noisy samples, ensuring that the training set contains representative and diverse code samples that effectively capture stylistic patterns.

2) Supervised Contrastive Learning and Classification Training Process: We employ the Supervised Contrastive Learning (SupCon) framework in the embedding-based classification module of CGSD, dividing the overall training process into two stages: Supervised Contrastive Pre-training and Classification Fine-tuning. As shown in the left panel of Figure 1, the model architecture consists of an encoder f_θ , a projection head p_θ , and a classification head h_θ .

In the first stage, we perform supervised contrastive learning to enhance the intra-class compactness and inter-class separability of the representations. Given an input sample x_i , the encoder $f_\theta(\cdot)$ maps it to a hidden representation, which is then projected into a contrastive feature space by the projection head $p_\theta(\cdot)$:

$$\mathbf{z}_i = p_\theta \circ f_\theta(x_i), \quad (1)$$

where $\mathbf{z}_i$ is normalized to a unit vector.

For a given sample i , let $P(i)$ denote the set of positive samples (code samples generated by the same model) and $N(i)$ denote the set of negative samples (code samples generated by different models). The supervised contrastive loss is defined as:

$$\mathcal{L}_{\text{sup-CL}} = - \sum_{i=1}^N \frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\mathbf{z}_i^\top \mathbf{z}_p / \tau)}{\sum_{a \in P(i) \cup N(i)} \exp(\mathbf{z}_i^\top \mathbf{z}_a / \tau)}, \quad (2)$$

where τ is a temperature parameter that controls the smoothness of the distribution. During this stage, only the encoder f_θ and the projection head p_θ are updated. This training encourages samples from the same model to cluster more tightly in the contrastive space, thereby producing more discriminative feature representations for the subsequent classification task.

Stage 2: Classification Fine-tuning. After contrastive pre-training, the projection head p_θ is removed. The encoder f_θ is either frozen or fine-tuned, and a classification head $h_\theta(\cdot)$ is added. For a given input x_i , the prediction probability for class c is:

$$\hat{y}_{i,c} = \text{softmax}(h_\theta \circ f_\theta(x_i)), \quad (3)$$

and the optimization objective is the cross-entropy loss:

$$\mathcal{L}_{\text{cls}} = - \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c}, \quad (4)$$

where $y_{i,c}$ denotes the one-hot encoded ground-truth label. The final classifier learns class-discriminative decision boundaries by minimizing $\mathcal{L}_{\text{cls}}$.

3) Evaluation and Analysis: After training, we evaluate the model on a held-out test set, measuring classification accuracy as the primary metric. Additionally, we save the predicted logits and true labels for further analysis, identifying both high-confidence and low-confidence misclassifications to understand the model’s behavior and potential failure modes. The projection vectors are exported to support visualization and downstream analysis of the learned representations.

B. Code Similarity Analysis

To compare stylistic similarities of code from different LLMs, we employ TF-IDF vectorization [10], which combines local term frequency (TF) with global inverse document frequency (IDF):

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \text{IDF}(t), \quad (5)$$

where

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \quad \text{IDF}(t) = \log \frac{|D|}{|\{d \in D : t \in d\}|}. \quad (6)$$

Here, $f_{t,d}$ is the frequency of term t in document d , $|D|$ is the total number of documents, and $|\{d \in D : t \in d\}|$ counts documents containing t .

We extract both character-level n -grams (3,3), (4,4), (3,5) and word-level n -grams (1,1), (1,2). Cosine similarity between vectorized snippets measures stylistic convergence:

$$\text{cosine}(u, v) = \frac{u \cdot v}{\|u\| \|v\|} \in [-1, 1]. \quad (7)$$

Additionally, we analyze variable naming by lexically extracting identifiers (excluding keywords and built-ins) and computing Top- k frequent distributions to capture model-specific naming tendencies.

C. Code Style Analysis

To systematically evaluate LLM code style, we develop Code Style Analyzer, an automated tool for static feature extraction and style quantification. It covers six core dimensions:

- 1) **Naming:** identifier naming conventions (camelCase, snake_case, PascalCase, UPPERCASE, lowercase), average name length, and single-character variable usage.
- 2) **Comments:** frequency and density of comments (single-line, block, and docstrings).
- 3) **Syntax:** structural indicators including number of functions/classes, control-flow statements such as `if`, `for`, `while`, and `try-except`, indentation style, line length, and blank line distribution.
- 4) **Consistency:** Average spacing in code, including operator spacing and spacing around punctuation, reflecting coding consistency within and across files.
- 5) **Complexity:** logical complexity via cyclomatic complexity [11], maximum nesting depth, function length, and code size. Cyclomatic complexity is defined as:

$$M = E - N + 2P, \quad (8)$$

where E is the number of edges, N the number of nodes, and P the number of connected components (typically $P = 1$). Higher values indicate more complex and less maintainable code.

- 6) **Patterns:** coding habits including library preferences, exception handling strategies, and function return styles including early return and empty return.

III. EXPERIMENTS AND RESULTS

This work aims to evaluate the robustness of code style detection across multiple large language models (LLMs). The primary task is a multi-class classification problem, where stylistic and structural cues in code are leveraged to distinguish the model of origin.

A. Dataset Construction

We construct a multi-class dataset tailored for style attribution by aggregating model responses to a shared pool of CodeAlpaca prompts [12], following the Alpaca instruction-tuning recipe [13] built on LLaMA [14] and Self-Instruct [15]. The resulting benchmark pairs completions from multiple mainstream LLMs on matched prompts so that stylistic differences arise from model behaviour rather than task mismatch.

B. Experimental Setup

We use LLM2Vec [16] as the base encoder, built on Meta-Llama-3-8B [17]. We attach a projection head for supervised contrastive learning to obtain a separable embedding space, and we fine-tune a classification head for M -way attribution. Inputs are truncated to 512 tokens for stability and efficiency, and features include textual embeddings and structural patterns (variable naming, function layout, comment usage). Under a unified setup, we report overall accuracy, visualize the learned separable projection, and run a loss ablation comparing supervised contrastive learning plus classification fine-tuning against single-objective baselines. Detailed configurations are in Appendix C.

C. Classification Results

To evaluate the effectiveness of our proposed CGSD framework in distinguishing between different LLM-generated code styles, we conducted a comprehensive classification experiment. The results demonstrate strong performance across various conditions.

Figure 2 compares UMAP projections from CGSD’s SupCon-trained projection and the LLM2Vec baseline. CGSD yields a more separable space, supporting downstream classification.

Table I presents classification accuracy comparisons. Our supervised contrastive learning followed by classification fine-tuning (CGSD) achieves 93.25%, significantly outperforming LLM2Vec (60.20%), BERT (58.00%), GPT-2 (32.75%), and T5 (28.25%). Complete training settings are detailed in Appendix C.

Beyond raw numbers, Table I highlights two trends. First, CGSD attains a +33.05% absolute gain over the strongest baseline (93.25 vs. 60.20 for LLM2Vec), corresponding to roughly

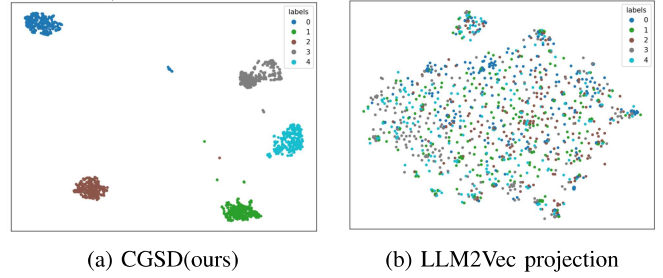


Fig. 2: Label separation in embedding space: CGSD SupCon (Left) forms distinct clusters, while LLM2Vec (Right) exhibits intermingling.

Model	Acc (%)
LLM2Vec	60.20
BERT	58.00
GPT-2	32.75
T5	28.25
CGSD (SupCon + CE)	93.25

TABLE I: Classification accuracy comparison across baseline encoders and our approach.

a 55% relative improvement and an *error-rate reduction* from 39.80% to 6.75% ($\approx 83\%$). Second, the margin over LLM2Vec indicates that supervised contrastive learning paired with classification fine-tuning yields more discriminative, attribution-ready embeddings than representation-only pretraining. These observations are consistent with our ablation in Table III, where SupCon+CE surpasses single-objective counterparts.

ChatGPT	Claude	DeepSeek	Gemini	Grok	Acc (%)
✓	✓				97.50
✓		✓			94.375
✓			✓		96.25
✓				✓	93.125
	✓	✓			94.375
	✓		✓		98.75
	✓			✓	93.75
		✓	✓		96.875
		✓		✓	90.625
			✓	✓	98.75

TABLE II: Binary classification accuracy across all pairwise model combinations.

To validate discriminative power, we run pairwise binary classification on ten model pairs using a balanced 2,000-sample dataset (400 per class). Each pair trains for 3 epochs with 640/160 train/test samples. Table II summarizes the accuracies. All pairs exceed 90% accuracy, with the best (Claude–Gemini, Gemini–Grok) at 98.75%, confirming that supervised contrastive learning reliably separates model-specific styles.

D. Ablation: Loss Configuration

We analyze the contribution of each training objective through an ablation study. Table III presents the results.

The full combination of supervised contrastive learning with classification fine-tuning (SupCon + CE) achieves 93.25%, outperforming CE alone (60.20%) by 33 percentage points and SupCon alone (92.25%). This confirms that both contrastive pre-training and classification fine-tuning are essential for effective source attribution.

Training Objective	Acc (%)
SupCon + CE (full)	93.25
CE only	60.20
SupCon only	92.25

TABLE III: Ablation study on training objectives.

E. Code Similarity Analysis

1) Overall Similarity Across Different n -gram Methods:

We analyzed the similarity of code generated by different LLMs using five n -gram feature settings. The overall similarity statistics are shown in Table IV, and they confirm that even the closest character-based profiles stay below 0.05 cosine similarity—indicating that stylistic overlap remains limited despite operating on the same prompts.

Method	Mean	Min	Max	Std
char_ngram3-3	0.0370	0.0348	0.0429	0.0028
char_ngram3-5	0.0188	0.0170	0.0206	0.0013
char_ngram4-4	0.0160	0.0146	0.0178	0.0011
word_ngram1-1	0.0206	0.0184	0.0230	0.0016
word_ngram1-2	0.0096	0.0085	0.0106	0.0007

TABLE IV: Overall similarity across n -gram methods.

2) *Similarity Between Models:* Heatmap visualizations showing n -gram similarity patterns across models are provided in Appendix A. The visualizations employ an adaptive color mapping strategy to improve readability and highlight that Grok most often clusters with ChatGPT, whereas Claude still stands out as the most stylistically distinct model overall.

3) *Code Top-K Variable Analysis:* We conducted frequency statistics on variables used in the generated code. Visualizations of the top 10 most frequent variables across models are provided in Appendix B that highlight how closely different models align in their variable naming habits.

F. Code Style Analysis Results

We systematically analyzed code style across six dimensions: naming, comments, syntax, consistency, complexity, and patterns. Detailed results are provided in Appendix D.

1) Naming Style & Syntax Structure

Figure 3 and Figure 4 compare the distribution of naming styles and syntax structures across different models, respectively. ChatGPT tends toward descriptive identifiers, while Gemini prefers terse tokens. Regarding syntax structures, Claude most frequently uses try-except statements and exhibits the most robust exception handling patterns.

2) Commenting Habits & Coding Consistency

Figure 5 presents the comparison of comment density

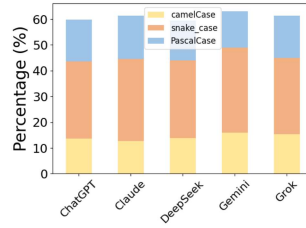


Fig. 3: Comparison of naming style across models.

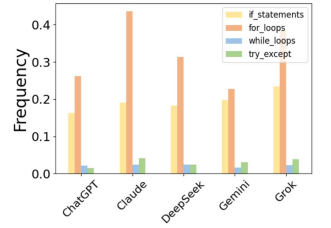


Fig. 4: Comparison of syntax structure across models.

among models, while Figure 6 shows each model’s consistency in operator spacing. Gemini tends to write more readable comments. Grok exhibits the most consistent style choices regarding quotes, operator spacing, and other formatting conventions.

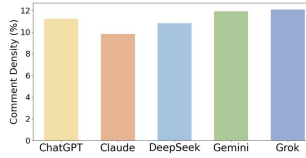


Fig. 5: Comparison of comment density across models.

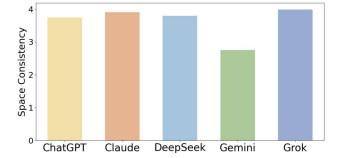


Fig. 6: Comparison of coding consistency across models.

3) Code Complexity

Code Style Analyzer metrics on cyclomatic complexity, maximum nesting depth, and average function length show that Grok exhibits the highest branching and deeper nesting, while ChatGPT favors flatter structures that are easier to audit.

4) Programming Patterns & Library Usage

Analysis of programming patterns, including library usage, exception handling, and early-return habits, shows that Claude demonstrates the broadest API coverage and returns immediately when exceptions or special conditions are detected, whereas DeepSeek and ChatGPT focus on built-in primitives and typically return default values.

Such cross-dimension patterns provide discriminative evidence for CGSD and highlight maintainability risks tied to specific model lineages.

IV. DISCUSSION

Supervised contrastive learning followed by classification fine-tuning enables CGSD to outperform baseline encoders by establishing a separable embedding space where stylistically similar samples cluster together. The ablation study confirms that both training stages are essential, with the two-stage approach effectively capturing discriminative stylistic features that single-objective training fails to learn.

Our analysis reveals that different LLMs produce code with distinct stylistic fingerprints despite architectural similarities. Character-level n -grams indicate shared syntax structures,

while word-level features show greater divergence. Model-specific preferences emerge in naming, comments, and control-flow patterns: ChatGPT favors descriptive identifiers such as `number` or `square`, while Gemini prefers terse tokens and exhibits lower comment density; Grok shows higher comment density and more consistent formatting. These patterns suggest intrinsic generative behaviors robust to surface-level variations.

These findings have practical implications for code provenance, quality assessment, security auditing, and model selection in software development.

V. CONCLUSION

We studied stylistic idiosyncrasies in LLM-generated code and introduced the Code Generation Source Detection (CGSD) framework for attribution-aware analysis. CGSD attains 93.25% accuracy, outperforming LLM2Vec (60.20%) and BERT (58.00%), and ablating the supervised contrastive stage drops performance to 60.20%, confirming the need for contrastive pre-training before classification fine-tuning. The learned embedding space keeps stylistically aligned samples close while preserving separability across model lineages, revealing consistent differences in naming, commenting, syntax, consistency, and complexity. These results show that LLM-produced programs retain stable fingerprints that enable reliable provenance tracking and illuminate maintainability and security risks. We will release our dataset together with reproducible training and analysis scripts to support deployment and future work on accountable AI-driven software development.

REFERENCES

- [1] Tristan Coignion and Quinton et.al. A Performance Study of LLM-Generated Code on Leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2024.
- [2] Zeeshan Rasheed, Malik Abdul Sami, Kai-Kristian Kemell, Muhammad Waseem, Mika Saari, Kari Systä, and Pekka Abrahamsson. CodePori: Large-Scale System for Autonomous Software Development Using Multi-Agent Technology, 2024.
- [3] Yuji Fu, Liang Peng, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. Security Weaknesses of Copilot Generated Code in GitHub. *arXiv preprint arXiv:2310.02059*, 2023.
- [4] Xianjun Yang, Kexun Zhang, Haifeng Chen, Linda Petzold, William Yang Wang, and Wei Cheng. Zero-Shot Detection of Machine-Generated Codes. *arXiv preprint arXiv:2310.05103*, 2023.
- [5] Guangsheng Bao, Yanbin Zhao, Zhiyang Teng, Linyi Yang, and Yue Zhang. Fast-DetectGPT: Efficient Zero-Shot Detection of Machine-Generated Text via Conditional Probability Curvature. *arXiv preprint arXiv:2310.05130*, 2023.
- [6] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. DetectGPT: Zero-Shot Machine-Generated Text Detection Using Probability Curvature. *arXiv preprint arXiv:2301.11305*, 2023.
- [7] Mingjie Sun, Yida Yin, Zhiqiu Xu, J. Zico Kolter, and Zhuang Liu. Idiosyncrasies in Large Language Models. *arXiv preprint arXiv:2502.12150*, 2025.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, and Kaiser et.al. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [9] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners, 2019.
- [10] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5):513–523, 1988.
- [11] Thomas J McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, (4):308–320, 1976.
- [12] Sahil Chaudhary. Code Alpaca: An Instruction-following LLaMA model for code generation. <https://github.com/sahil280114/codealpaca>, 2023.
- [13] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [14] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*, 2023.
- [15] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, and Daniel et.al Khashabi. Self-Instruct: Aligning Language Models with Self-Generated Instructions. *arXiv preprint arXiv:2212.10560*, 2023.
- [16] Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders. *arXiv preprint arXiv:2404.05961*, 2024.
- [17] Meta AI. Llama 3 model card. <https://ai.meta.com/llama>, 2024.
- [18] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A Simple Framework for Contrastive Learning of Visual Representations. *arXiv preprint arXiv:2002.05709*, 2020.
- [19] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschiot, Ce Liu, and Dilip Krishnan. Supervised Contrastive Learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:18661–18673, 2020.

APPENDIX A N-GRAM SIMILARITY VISUALIZATIONS

This appendix presents pairwise TF-IDF n-gram similarity heatmaps across models, providing an intuitive view of lexical overlap and divergence patterns. These visualizations complement Section III-E by offering detailed pairwise comparisons showing how different LLM-generated code samples share common n-gram patterns versus exhibit distinctive signatures.

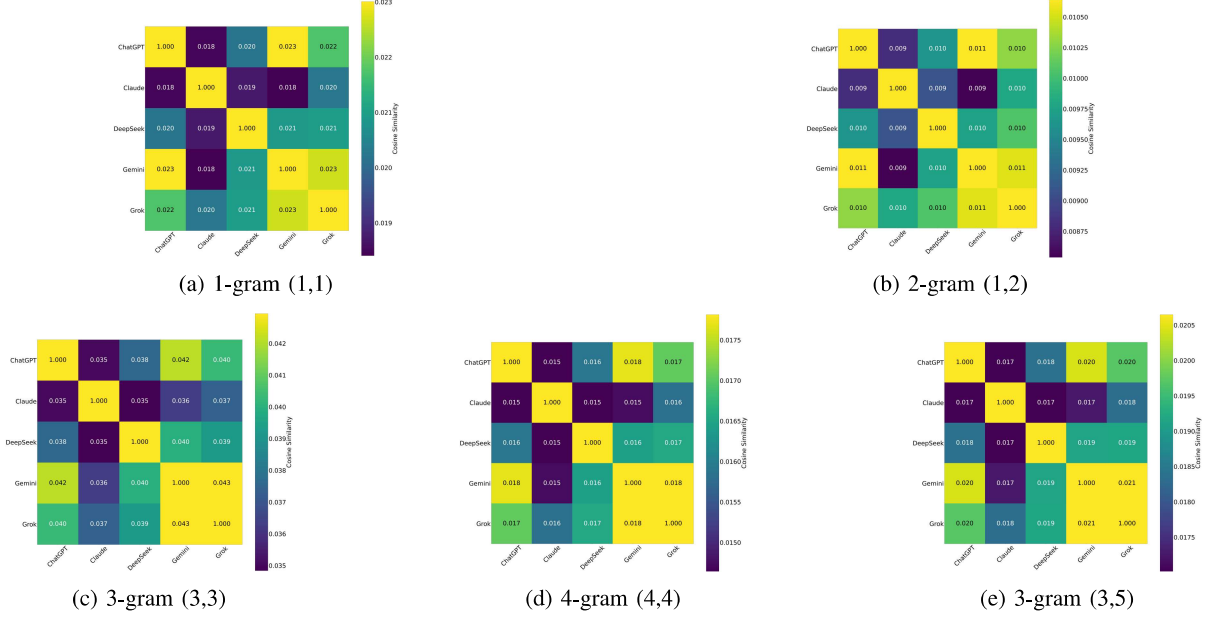


Fig. 7: Heatmap visualizations of n-gram similarity patterns across models.

APPENDIX B VARIABLE FREQUENCY VISUALIZATIONS

This appendix reports the top-k variable tokens per model to highlight frequent naming conventions and surface-level stylistic tendencies. These visualizations extend Section III-E by analyzing variable naming patterns across different LLMs, revealing model-specific preferences in identifier choices.

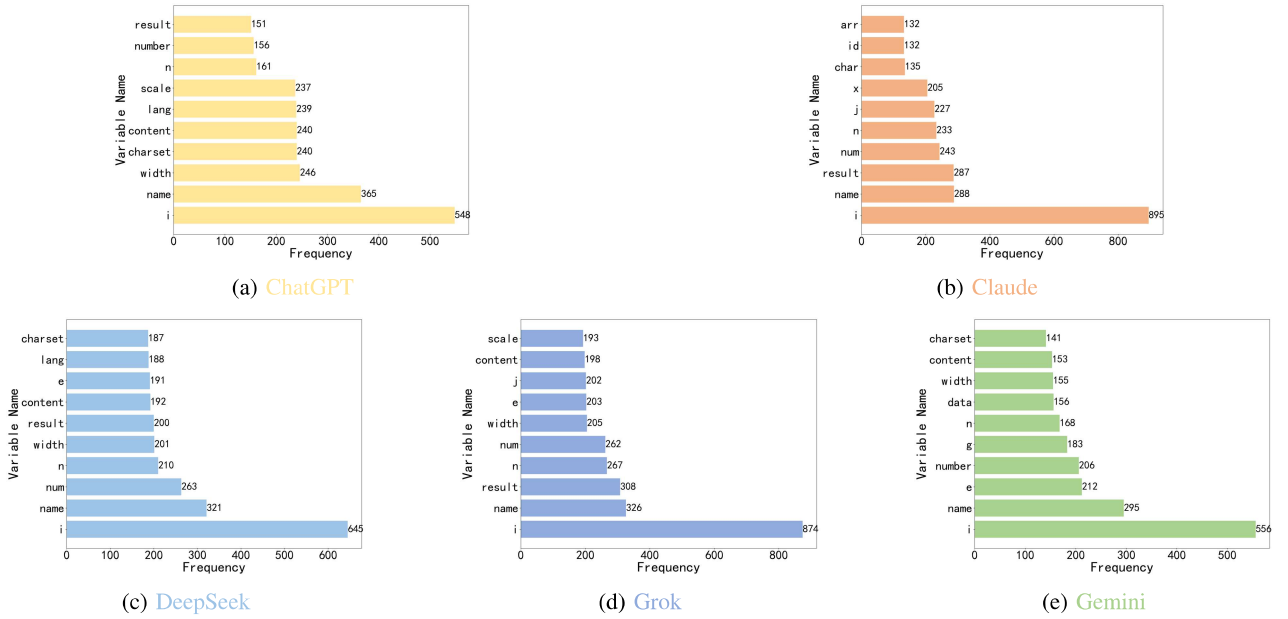


Fig. 8: Top 10 most frequent variables across models.

APPENDIX C

IMPLEMENTATION DETAILS

Response Generation

We report our procedure for generating responses from different large language models (LLMs). For chat-based LLM APIs, we accessed the latest stable releases of each model via official endpoints between July 1, 2025, and October 15, 2025. Responses were generated with default sampling settings. For instruct LLMs, we used greedy decoding to sample outputs. For base LLMs, we set the temperature to $T = 0.6$ and applied a repetition penalty of 1.1 to reduce repetitive completions.

Training Setup

We describe the contrastive-classification learning process on LLM-generated code samples. For data preparation, we employed a data cleaning strategy to select high-quality samples from multiple response files, ensuring representative and diverse training data. We used the first 512 tokens of each generated response for training and evaluation.

Our model is based on LLM2vec [16], which uses Meta-Llama-3-8B [17] with fine-tuning and adapter components. We added two components: a classification head for M -way classification and a projection head for contrastive learning [18]. The contrastive-classification training consists of the following stages:

Stage 1 (Supervised Contrastive Pre-training): We train the projection head and optionally the encoder using supervised contrastive loss (SupCon) [19]. This stage clusters samples from the same class (positive samples) together while separating samples from different classes (negative samples) in the embedding space.

Stage 2 (Classification Fine-tuning): We fine-tune using cross-entropy (CE) loss with only the classification head being trainable while the encoder remains frozen. Table V summarizes the training configuration.

During evaluation, we measure classification accuracy and save predicted logits and true labels for analysis. We identify both high-confidence and low-confidence misclassifications and export projection vectors for visualization.

For the main baseline comparison (Table I), results are averaged over three runs. For the ablation study (Table III), results are averaged over six runs to ensure statistical stability.

Config	Value
Optimizer	AdamW
Weight decay	0.001
Optimizer momentum	$\beta_1, \beta_2 = 0.9, 0.999$
Training epochs	3 / 6
Batch size	8
Learning rate schedule	cosine decay
Warmup schedule	linear warmup, ratio 10%
Gradient clipping	0.3

TABLE V: Fine-tuning configuration for text embedding models on LLM responses.

APPENDIX D
DETAILED CODE STYLE STATISTICS

This appendix presents the full version of the code style statistics, including all dimensions and model-level comparisons.

Dimension	Metric	ChatGPT	Claude	DeepSeek	Gemini	Grok
Naming	naming variables count	2.457	3.261	2.772	2.440	3.191
	naming variables cases lowercase	2.038	2.602	2.231	1.774	2.544
	naming variables cases Pascal Case	0.402	0.549	0.428	0.344	0.522
	naming variables cases Snake Case	0.733	1.036	0.837	0.804	0.948
	naming variables cases camel Case	0.337	0.415	0.384	0.389	0.487
	naming variables cases UPPERCASE	0.040	0.036	0.053	0.039	0.036
	naming variables length avg	4.713	4.305	4.346	4.089	4.309
	naming variables length min	3.515	3.054	3.145	2.852	3.015
	naming variables length max	6.170	5.900	5.879	5.591	5.980
	naming variables Single Char Count	0.255	0.355	0.309	0.256	0.368
	naming functions count	1.050	1.528	1.007	1.099	1.235
	naming functions cases camel Case	0.249	0.378	0.240	0.218	0.355
	naming functions cases Snake Case	0.624	0.924	0.549	0.596	0.608
	naming functions cases Pascal Case	0.015	0.027	0.017	0.046	0.022
	naming functions cases UPPERCASE	0.000	0.000	0.001	0.000	0.000
	naming functions cases lowercase	0.228	0.350	0.239	0.211	0.282
	naming functions length avg	5.233	5.084	4.503	5.623	4.282
	naming functions length min	5.063	4.716	4.360	5.458	3.951
	naming functions length max	5.419	5.481	4.662	5.821	4.638
	naming functions Single Char Count	0.000	0.000	0.000	0.000	0.000
	naming classes count	0.329	0.333	0.287	0.330	0.359
	naming classes cases Pascal Case	0.257	0.281	0.247	0.218	0.273
	naming classes cases camel Case	0.000	0.000	0.000	0.000	0.000
	naming classes cases Snake Case	0.002	0.00039	0.00078	0.005	0.00079
	naming classes cases UPPERCASE	0.000	0.000	0.000	0.000	0.000
	naming classes cases lowercase	0.067	0.035	0.031	0.087	0.072
	naming classes length avg	1.219	1.027	1.136	1.098	1.072
	naming classes length min	1.172	0.967	1.087	1.018	1.015
	naming classes length max	1.269	1.093	1.190	1.185	1.144
	naming classes Single Char Count	0.000	0.000	0.000	0.000	0.000
Comments	comments Total Comments	1.019	1.416	1.181	1.881	1.745
	comments Single Line Comments	0.914	1.313	1.013	1.611	1.596
	comments Python Comments	0.587	0.846	0.563	0.747	0.909
	comments Block Comments	0.039	0.027	0.038	0.115	0.048
	comments docstrings	0.066	0.076	0.131	0.155	0.101
	comments Comment Density	11.225	9.827	10.810	11.918	12.080
Syntax	syntax Functions Count	0.431	0.635	0.423	0.415	0.471
	syntax Classes Count	0.151	0.150	0.134	0.155	0.156
	syntax Control Flow If Statements	0.163	0.191	0.182	0.198	0.235
	syntax Control Flow For Loops	0.261	0.435	0.313	0.227	0.384
	syntax Control Flow While Loops	0.022	0.025	0.025	0.017	0.023
	syntax Control Flow Try Except	0.016	0.042	0.025	0.031	0.039
	syntax indentation Avg Spaces	3.741	3.904	3.794	2.749	3.986
	syntax indentation Avg Tabs	0.001	0.00045	0.001	0.002	0.00068
	syntax indentation Mixes Tabs Spaces	0.000	0.000	0.000	0.00018	0.000
	syntax Line Length avg	35.709	28.100	30.915	37.254	34.210
	syntax Line Length max	68.720	52.492	64.408	81.115	74.978
	syntax Empty Lines	0.444	1.278	0.929	0.958	1.210

TABLE VI: Code style metrics comparison across LLMs (Part 1).

Dimension	Metric	ChatGPT	Claude	DeepSeek	Gemini	Grok
Consistency	consistency quotes Single Quotes	0.585	0.917	0.806	0.850	0.894
	consistency quotes Double Quotes	1.581	1.860	2.063	2.600	2.428
	consistency Operator Spacing + With Spaces	0.353	0.408	0.439	0.411	0.511
	consistency Operator Spacing + Without Spaces	0.199	0.300	0.264	0.229	0.316
	consistency Operator Spacing - With Spaces	0.486	0.216	0.878	0.206	0.810
	consistency Operator Spacing - Without Spaces	0.894	0.998	1.679	2.150	1.737
	consistency Operator Spacing * With Spaces	0.106	0.195	0.177	1.300	0.204
	consistency Operator Spacing * Without Spaces	0.743	0.083	1.938	2.218	1.162
	consistency Operator Spacing / With Spaces	0.032	0.047	0.048	0.051	0.055
	consistency Operator Spacing / Without Spaces	0.801	0.732	0.835	0.871	0.800
	consistency Operator Spacing = With Spaces	1.635	2.357	2.025	1.820	2.338
	consistency Operator Spacing = Without Spaces	0.758	0.641	0.769	0.625	0.808
	consistency Operator Spacing == With Spaces	0.140	0.217	0.168	0.127	0.206
	consistency Operator Spacing == Without Spaces	0.007	0.008	0.018	0.018	0.020
	consistency Operator Spacing != With Spaces	0.037	0.053	0.042	0.032	0.053
	consistency Operator Spacing != Without Spaces	0.00018	0.00017	0.00017	0.00054	0.00017
	consistency Operator Spacing >With Spaces	0.075	0.107	0.099	0.076	0.113
	consistency Operator Spacing >Without Spaces	0.508	0.394	0.479	0.470	0.449
	consistency Operator Spacing <With Spaces	0.119	0.161	0.132	0.130	0.181
	consistency Operator Spacing <Without Spaces	0.481	0.371	0.449	0.483	0.422
	consistency Operator Spacing >= With Spaces	0.025	0.037	0.031	0.031	0.047
	consistency Operator Spacing >= Without Spaces	0.00018	0.00035	0.00035	0.00018	0.00017
	consistency Operator Spacing <= With Spaces	0.067	0.094	0.087	0.076	0.100
	consistency Operator Spacing <= Without Spaces	0.00073	0.001	0.00035	0.00054	0.00070
Complexity	consistency containers Multi Line	0.808	1.004	1.043	0.829	1.160
	consistency containers Trailing Commas	0.007	0.010	0.010	0.007	0.009
	complexity Cyclomatic Complexity	1.355	1.437	1.419	1.432	1.542
	complexity Max Nesting Depth	1.213	1.313	1.294	1.092	1.335
Patterns	complexity Total Lines	9.371	11.900	12.252	12.837	13.726
	complexity Avg Function Lines	0.313	0.799	0.434	0.458	0.676
	patterns Library Calls numpy	0.062	0.144	0.080	0.055	0.122
	patterns Library Calls pandas	0.019	0.040	0.030	0.024	0.040
	patterns Library Calls sklearn	0.000	0.000	0.000	0.000	0.000
	patterns Library Calls matplotlib	0.005	0.014	0.011	0.010	0.021
	patterns Library Calls tensorflow	0.005	0.010	0.008	0.006	0.011
	patterns Library Calls pytorch	0.002	0.010	0.009	0.002	0.004
	patterns Library Calls requests	0.002	0.010	0.010	0.008	0.008
	patterns Library Calls React	0.004	0.003	0.009	0.002	0.004
	patterns Library Calls jQuery	0.000	0.003	0.003	0.000	0.003
	patterns Library Calls axios	0.002	0.003	0.002	0.000	0.001
	patterns Library Calls express	0.000	0.007	0.002	0.000	0.004
	patterns Library Calls vue	0.000	0.001	0.000	0.000	0.000
	patterns Library Calls angular	0.000	0.000	0.002	0.002	0.000
	patterns Library Calls stdio	0.364	0.762	0.519	0.412	0.688
	patterns Library Calls stdlib	0.061	0.190	0.111	0.098	0.250
	patterns Library Calls string	0.478	1.457	0.667	0.905	1.047
	patterns Library Calls std	0.000	0.000	0.000	0.007	0.000
	patterns Library Calls serde	0.000	0.000	0.000	0.000	0.000
	patterns Library Calls tokio	0.000	0.000	0.000	0.000	0.000
	patterns Library Calls Stream	0.000	0.000	0.000	0.000	0.000
	patterns Library Calls List	0.000	0.000	0.000	0.016	0.000
	patterns Library Calls Map	0.000	0.000	0.000	0.000	0.031
	patterns Library Calls Array List	0.000	0.000	0.000	0.000	0.000
	patterns Library Calls Hash Map	0.000	0.000	0.000	0.000	0.000
	patterns Library Calls System	3.111	2.667	2.333	2.968	2.906
	patterns Library Calls iostream	0.769	0.840	0.792	0.826	0.812
	patterns Library Calls vector	0.385	0.680	0.542	0.391	0.375
	patterns Library Calls algorithm	0.231	0.000	0.167	0.130	0.062
	patterns Library Calls fmt	0.000	0.400	0.167	0.333	0.333
	patterns Library Calls net/http	0.000	0.000	0.000	0.000	0.000
	patterns Library Calls encoding/json	0.000	0.000	0.000	0.000	0.000
	patterns Try Except Count Value	0.011	0.034	0.020	0.022	0.031
	patterns Early Returns Value	0.511	0.879	0.625	0.414	0.754
	patterns None Returns Value	0.012	0.024	0.018	0.017	0.017

TABLE VII: Code style metrics comparison across LLMs (Part 2).